

CALC-X — Advanced Terminal Calculator ■

A menu-driven, terminal-based mathematical calculator built using **Python 3**. CALC-X provides a collection of mathematical tools, including basic arithmetic, scientific calculations, algebra, equation solving, matrix operations, statistics, calculus, and calculation history.

The project follows a **beginner-friendly procedural programming approach** without custom classes. It also features a manually implemented expression evaluator that uses tokenization and recursive-descent parsing instead of Python's built-in `eval()` function.

■ Table of Contents

- About the Project
 - Features
 - Technologies Used
 - Project Structure
 - Prerequisites
 - Installation
 - How to Run
 - Main Menu
 - Module Descriptions
 - Expression Evaluation
 - Calculation History
 - Error Handling
 - Example Usage
 - Limitations
 - Future Enhancements
 - Learning Outcomes
 - Contributing
 - License
-

■ About the Project

CALC-X is an advanced command-line calculator designed to bring multiple mathematical operations together in one application.

Instead of creating separate programs for different mathematical problems, CALC-X provides a single menu-driven interface through which users can select the required operation.

The project is organized into independent Python modules. Each module is responsible for a particular category of mathematical functionality, making the code easier to understand, maintain, test, and extend.

Project Objectives

- Provide a single interface for different mathematical calculations.
 - Implement mathematical expression parsing without Python's `eval()`.
 - Solve algebraic equations and simultaneous linear equations.
 - Perform matrix and statistical operations.
 - Support polynomial differentiation and integration.
 - Maintain calculation history for supported calculator operations.
 - Demonstrate modular programming and procedural problem-solving in Python.
-

■ Features

1. Basic Calculator

- Evaluate arithmetic expressions.
- Perform addition, subtraction, multiplication, division, and modulus operations.
- Handle brackets and operator precedence.
- Validate mathematical expressions before evaluation.

2. Scientific Calculator

- Evaluate supported scientific mathematical expressions.
- Use mathematical functions and powers supported by the expression parser.
- Apply operator precedence when evaluating expressions.

The exact supported functions and syntax depend on the implementation in `calculator.py`.

3. Algebra Solver

- Solve linear equations.
- Solve quadratic equations.
- Evaluate polynomials for given values.

4. Equation Solver

- Solve simultaneous linear equations involving two variables.
- Solve simultaneous linear equations involving three variables.

5. Matrix Operations

- Matrix addition.
- Matrix subtraction.
- Matrix multiplication.
- Matrix transpose.
- Matrix determinant.
- Matrix inverse.

6. Statistics

- Mean.
- Median.
- Mode.
- Minimum and maximum values.
- Range.
- Variance.
- Standard deviation.

7. Calculus

- Polynomial differentiation.
- Polynomial integration.

8. Calculation History

- Add supported calculations to history.
- View previous calculations.
- Save history to a file.
- Load previously saved history.
- Clear calculation history.

Note: According to the current project design, automatic calculation-history recording is supported by the Basic and Scientific Calculator modules. The History module also provides the history-management interface.

■ Technologies Used

Technology	Purpose
Python 3	Main programming language
Standard Python libraries	Input handling, mathematical processing, and file operations where implemented
Command-line interface (CLI)	Menu navigation and user interaction
Procedural programming	Organizing the application into functions
Recursive-descent parsing	Parsing and evaluating mathematical expressions
File handling	Saving and loading history, where implemented

No graphical user interface is required. The application runs directly in a terminal.

■ Project Structure

```
CALC-X/  
■  
■■■ main.py  
■■■ calculator.py  
■■■ algebra.py  
■■■ equations.py  
■■■ matrix.py
```

```
■■■ statistics.py
■■■ calculus.py
■■■ history.py
■■■ README.md
```

File Responsibilities

File	Responsibility
<code>main.py</code>	Main menu, navigation, prompts, and user interaction
<code>calculator.py</code>	Expression tokenization, validation, parsing, and evaluation
<code>algebra.py</code>	Linear equations, quadratic equations, and polynomial evaluation
<code>equations.py</code>	Simultaneous 2×2 and 3×3 linear equation solving
<code>matrix.py</code>	Matrix arithmetic and matrix-related operations
<code>statistics.py</code>	Statistical calculations and descriptive measures
<code>calculus.py</code>	Polynomial differentiation and integration
<code>history.py</code>	Adding, viewing, saving, loading, and clearing history
<code>README.md</code>	Project documentation and instructions

■ Prerequisites

Before running CALC-X, make sure you have:

- Python 3 installed on your computer.
- A terminal, Command Prompt, PowerShell, or compatible IDE terminal.
- The complete CALC-X project folder with all required Python modules.

Check your Python installation:

```
python --version
```

On some systems, you may need to use:

```
python3 --version
```

■ Installation

Step 1: Clone the Repository

Replace **YOUR-USERNAME** with your GitHub username and **YOUR-REPOSITORY** with your actual repository name.

```
git clone https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git
```

Step 2: Open the Project Folder

```
cd YOUR-REPOSITORY
```

If your repository contains the **CALC-X** folder inside it, enter that folder:

```
cd CALC-X
```

If the repository itself is the CALC-X folder, you do not need the second command.

Step 3: Verify Python

```
python --version
```

Use `python3` instead of `python` if that is the command configured on your system.

■ ■ How to Run

From inside the folder containing `main.py`, execute:

```
python main.py
```

Alternatively:

```
python3 main.py
```

The application will display the main menu in the terminal.

Follow the prompts to select a calculator module, enter the required values or expressions, and view the results.

■ ■ Main Menu

The main menu provides nine options:

```
===== CALC-X =====
Advanced Terminal Calculator

1. Basic Calculator
2. Scientific Calculator
3. Algebra Solver
4. Equation Solver
5. Matrix Operations
6. Statistics
7. Calculus
8. History
9. Exit
```

Menu Options

Option	Module	Description
1	Basic Calculator	Evaluates basic arithmetic expressions
2	Scientific Calculator	Evaluates supported scientific expressions
3	Algebra Solver	Solves supported algebraic problems
4	Equation Solver	Solves simultaneous linear equations
5	Matrix Operations	Performs matrix calculations
6	Statistics	Calculates statistical measures

Option	Module	Description
7	Calculus	Differentiates and integrates polynomials
8	History	Manages saved and previous calculations
9	Exit	Terminates the application

■ Module Descriptions

`main.py` — Main Application

This file acts as the entry point of CALC-X.

Its responsibilities include:

- Displaying the main menu.
- Accepting the user's menu selection.
- Calling the appropriate module functions.
- Managing navigation between the menu and calculator modules.
- Passing the history list to the calculator modules that support history recording.
- Allowing the user to exit the application.

Run this file to start the application.

`calculator.py` — Expression Calculator

This module handles mathematical expression processing.

Its responsibilities include:

- Tokenizing input expressions.
- Validating expression structure.
- Parsing mathematical expressions.
- Evaluating supported arithmetic and scientific operations.
- Applying operator precedence.
- Reporting invalid expressions and mathematical errors.

The expression evaluator is implemented without Python's built-in `eval()` function.

`algebra.py` — Algebra Solver

This module provides tools for common algebraic problems.

Its intended operations include:

- Solving linear equations.
- Solving quadratic equations.
- Evaluating polynomials for user-supplied values.

equations.py — Simultaneous Equation Solver

This module handles systems of linear equations involving two or three variables.

For example, a two-variable system may be represented as:

$$\begin{array}{rcl} 2x + y & = & 7 \\ x - y & = & 2 \end{array}$$

The solver calculates values for the unknown variables when the system has a unique solution.

Systems with no unique solution require appropriate handling by the implementation.

matrix.py — Matrix Calculator

This module performs common matrix operations:

- Addition and subtraction.
- Multiplication.
- Transpose.
- Determinant.
- Inverse.

Matrix operations may require compatible matrix dimensions. Determinants and inverses are only defined for square matrices, and an inverse exists only when the matrix is nonsingular.

statistics.py — Statistical Analyzer

This module calculates descriptive statistics for numerical data.

Available operations include:

- Mean: the arithmetic average.
- Median: the middle value after sorting.
- Mode: the most frequently occurring value or values, depending on the implementation.
- Minimum: the smallest value.
- Maximum: the largest value.
- Range: the difference between the maximum and minimum.
- Variance: a measure of data dispersion.
- Standard deviation: the square root of variance.

The exact variance convention, such as population or sample variance, depends on the implementation.

calculus.py — Polynomial Calculus

This module performs polynomial differentiation and integration.

Example polynomial:

$$f(x) = 3x^2 + 2x + 5$$

Its derivative is:

$$f'(x) = 6x + 2$$

Its indefinite integral is:

$$\int f(x) dx = x^3 + x^2 + 5x + C$$

Here, **C** represents the constant of integration.

history.py — Calculation History

This module manages the application's calculation history.

Its responsibilities include:

- Adding calculation records.
- Displaying stored history.
- Saving history to a file.
- Loading saved history.
- Clearing history.

The history module provides history-management functionality, while automatic recording is currently connected to the Basic and Scientific Calculator modules.

■ Expression Evaluation

One of the key design features of CALC-X is its manual mathematical expression evaluator.

Rather than passing an expression directly to Python's `eval()`, the calculator processes the expression through tokenization and recursive-descent parsing.

How It Works

Step 1: Tokenization

The input expression is divided into meaningful tokens, such as numbers, operators, function names, and parentheses.

Example:

$$2 + 3 * 4$$

Conceptual tokens:

$$2, +, 3, *, 4$$

Step 2: Parsing

The parser processes tokens according to the grammar and precedence rules implemented in the calculator.

Step 3: Evaluation

The parsed expression is evaluated to obtain the result.

For example:

$$2 + 3 * 4 = 14$$

Multiplication is performed before addition.

Operator Precedence

The intended precedence order is:

- Parentheses.
- Supported mathematical functions.
- Exponentiation.
- Multiplication, division, and modulus.
- Addition and subtraction.

The exact behavior of chained powers, unary signs, and implicit multiplication depends on the parser's grammar.

Why Avoid `eval()`?

Using a custom parser gives the application greater control over which mathematical operations are accepted.

It also provides a useful learning opportunity for understanding:

- Lexical tokenization.
- Parsing and grammar.
- Operator precedence.
- Recursive functions.
- Mathematical expression evaluation.

A custom parser still requires careful validation and testing; avoiding `eval()` alone does not guarantee complete security or correctness.

■ Calculation History

CALC-X includes a history-management module for supported calculations.

Depending on the implemented functionality, users can:

- Record a calculation.
- View previously recorded calculations.
- Save history to persistent storage.
- Load history from storage.
- Clear the history list.

Current Recording Scope

Automatic history recording is currently supported in:

- Basic Calculator.
- Scientific Calculator.

The other modules do not automatically record their calculations unless history integration is added to their functions.

■ Error Handling

CALC-X is designed to handle invalid input and mathematical errors gracefully.

Examples of conditions that should be handled include:

- Invalid menu selections.
- Incorrectly formatted expressions.
- Division by zero.
- Missing or invalid numerical input.
- Incompatible matrix dimensions.
- Attempting to invert a singular matrix.
- Invalid equation formats.
- Empty datasets in statistical calculations.
- Invalid history file operations.

When an error occurs, the intended behavior is to display an understandable message and allow the user to try again instead of unexpectedly terminating the program.

Actual coverage depends on the error checks implemented in each module.

■ Example Usage

The following examples illustrate the types of calculations CALC-X is designed to support. They are mathematical examples, not screenshots or a transcript of the running application.

Example 1: Basic Arithmetic

Input:

$2 + 3 * 4$

Expected result:

14

Example 2: Quadratic Equation

Equation:

$$x^2 - 5x + 6 = 0$$

Expected roots:

$$\begin{aligned}x &= 2 \\x &= 3\end{aligned}$$

Example 3: Matrix Transpose

Input matrix:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Expected transpose:

$$\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Example 4: Statistical Mean

Dataset:

$$10, 20, 30, 40, 50$$

Expected mean:

$$30$$

Example 5: Polynomial Differentiation

Input:

$$f(x) = 3x^2 + 2x + 5$$

Expected derivative:

$$f'(x) = 6x + 2$$

■ ■ Limitations

- CALC-X is a terminal-based application and does not provide a graphical interface.
 - Supported mathematical functions depend on the expression parser's implementation.
 - Calculus operations are limited to the polynomial operations implemented in the project.
 - Equation solving is focused on supported linear systems and algebraic equations.
 - Matrix operations require valid dimensions and appropriate mathematical conditions.
 - Automatic history recording is currently limited to the Basic and Scientific Calculator modules.
 - Advanced symbolic mathematics, numerical optimization, and plotting are not included unless implemented separately.
-

■ Future Enhancements

Possible improvements for future versions include:

- Add more scientific functions, such as trigonometric and logarithmic functions.
- Extend equation solving to additional equation types.
- Add polynomial expansion and factorization.
- Support more advanced calculus operations.
- Add automatic history recording to every module.
- Export calculation history to CSV or JSON.
- Add unit conversion and complex-number operations.
- Add automated unit tests for mathematical functions and edge cases.
- Improve error messages and input validation.
- Add a graphical user interface in a future version.
- Add continuous integration to run tests automatically.

These are proposed enhancements and are not necessarily available in the current version.

■ Learning Outcomes

This project demonstrates practical application of:

- Python functions and procedural programming.
- Conditional statements and loops.
- Lists, dictionaries, and other built-in data structures.
- Input validation and exception handling.
- Modular programming and code organization.
- Recursive-descent parsing.
- Mathematical algorithms.
- File handling and persistent data.
- Debugging and testing.
- Designing menu-driven command-line applications.

CALC-X is suitable as a learning project for students exploring Python programming, mathematical algorithms, and software design.

■ Contributing

Contributions and suggestions are welcome.

To contribute:

- Fork the repository.
- Create a new branch for your changes.

- Implement your changes.
- Test the affected functionality.
- Commit your changes with a descriptive message.
- Open a pull request explaining the changes.

Please ensure that changes maintain the project's modular structure and do not break existing calculator functionality.

■ License

No license has been specified for this project yet.

If you plan to make CALC-X publicly available for reuse, choose an appropriate open-source license and add a **LICENSE** file to the repository.

■■■ Project Information

Project Name: CALC-X — Advanced Terminal Calculator

Language: Python 3

Application Type: Command-Line Interface (CLI)

Programming Approach: Procedural Programming

Primary Purpose: Mathematical calculations and algorithmic learning

CALC-X — One terminal. Multiple mathematical tools.